SDEV 2401

Rapid Backend App Development

REST API Development - 5

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments

- No Cheating

- Be a good classmate

- Don’t waste your time

- Show up to class

WE ARE ESSENTIAL TO ALBERTA | NAIT

Agenda

On the right is what we will cover today.

- Multiple Serializers for Related Models
- Custom Validation in Serializers
- Custom Permissions for User Data
- Filtering Querysets by User
- Nested Serializers for Rich Responses
- Key Takeaways

WE ARE ESSENTIAL TO ALBERTA
NAIT

Multiple Serializers for Related Models

- When building APIs with Django REST Framework (DRF), it's common to use different serializers for different actions.
- For example, you might use a simple serializer for creating/updating a `MealLog` and a more detailed one for viewing, which includes related data like the user and meal details.

```
class MealLogCreateSerializer(serializers.ModelSerializer):
    class Meta:
        model = MealLog
        fields = ['id', 'meal', 'calories', 'date']

class MealLogReadOnlySerializer(serializers.ModelSerializer):
    meal = MealSerializer(read_only=True)
    user = UserSerializer(read_only=True)
    class Meta:
        model = MealLog
        fields = ['id', 'meal', 'calories', 'date', 'user']
```

- This approach keeps input simple for creation, but provides rich detail for display.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Custom Validation in Serializers

- DRF allows you to add custom validation logic to serializers.
- For example, you might want to ensure that a `distance_km` field is only set for "running" activities, not for "yoga".

```
def validate(self, attrs):
    activity = attrs.get('activity')
    distance_km = attrs.get('distance_km')
    if activity == 'yoga' and distance_km:
        raise serializers.ValidationError("Distance should not be set for yoga activities.")
    return attrs
```

- This ensures your data remains consistent and meaningful.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Custom Permissions for User Data

- You can create custom permission classes to control who can modify or view certain data.
- For example, only allow users to edit their own `MealLog` entries, but let everyone view all logs.

```
class IsOwnerOrReadOnly(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        return obj.user == request.user
```

- This pattern is essential for multi-user applications to protect user data.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Filtering Querysets by User

- It’s common to show users only their own data in list views, but allow detail views for any item.
- For example, in a `MealLogViewSet`:

```
def get_queryset(self):
    if self.action == 'list':
        return MealLog.objects.filter(user=self.request.user)
    return MealLog.objects.all()
```

- This keeps list views relevant to the user, while still supporting broader access for detail views.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Nested Serializers for Rich Responses

- Use nested serializers to include related objects in API responses.
- For example, a `MealLogReadOnlySerializer` can include both the meal and user details:

```
class MealLogReadOnlySerializer(serializers.ModelSerializer):
    meal = MealSerializer(read_only=True)
    user = UserSerializer(read_only=True)
    class Meta:
        model = MealLog
        fields = ['id', 'meal', 'calories', 'date', 'user']
```

- This provides clients with all the information they need in a single API call.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Key Takeaways

- Use different serializers for create/update vs. read actions to simplify input and enrich output.
- Add custom validation to enforce business rules at the API level.
- Implement custom permissions to protect user data.
- Filter querysets to show users only their own data where appropriate.
- Use nested serializers for comprehensive, single-call responses.

Example

Let’s go do an example together